Lifelong Learning Techniques

for an Origami Robot

by

E. De Vroey

Student number: 4685156

TU Delft

Contents

1 Introduction

1.1. Application to soft (origami) robots

In the field of soft robotics, lifelong learning are particularly interesting.

1.2. Challenges

(Kim et al., 2021; Lesort et al., 2020).

> How can lifelong learning methods be employed to manage the evolving dynamics due to wear and tear in the control of a soft origami robot over its operational lifespan?

2

Background and definition

(Annaswamy & Fradkov, 2021; Landau et al., 2011; Lesort et al., 2020).

3

Blackbox methods

general intro, define ”blackbox”

3.1. Learning control policies

A perfect example of a blackbox method is given by Alessi et al. (2023). In their study they used Proximal Policy Optimization (PPO) (Schulman et al., 2017) to obtain a feedforward neural network (FNN) that can generalize over new dynamics and tasks. The controller is a blackbox: a desired trajectory, several measurements of the robots state, and other values of interest, such as the error of robot's end-effector tip, are used as inputs, resulting in a control action. It essentially acts in a manner akin to a feedback controller using an inverse dynamics model, only in this case it is unknown what is happening *under the hood*.

The previous example is *"perfect"* in the sense that it perfectly embodies the simplistic nature of blackbox methods, however it is worth noting that other techniques, such as more complicated architectures or combinations of multiple models, can be used in combination with these blackbox methods. Such architectural modifications will be discussed in chapter 4.

Apart from Alessi et al. (2023), several other examples of blackbox policies can be found in the literature.

In Zhou et al. (2022), two policies are learned for the control of a small humanoid robot. Exact details on the inputs and outputs of the controllers are not provided, but both policies are probabilistic in nature.

Thuruthel et al. (2019) use guided policy search (**<empty citation>**) over trajectory samples containing control actions for given regions in the state-space of a soft robotic manipulator. The resulting control policy is represented as a FNN which takes in current and previous states, as well as a desired target state, and returns the appropriate control action.

citation [17] from thuruthel2019

For the control of a soft robotic arm, in Nazeer et al. (2023), the policy is represented by a multi-layer perceptron (MLP) which takes in the arm's current state[1] and outputs a control action. The policy is trained using the Soft Actor-Critic (SAC) (**<empty citation>**) reïnforcement learning algorithm.

SAC reference

In Lu et al. (2020), a more complicated technique is used. A probabilistic 'skill' distribution $p_\psi(z|s)$ and a neural network-based policy $\pi_\theta(a|s,z)$ are independently and simultaneously learned using SAC. Both models are trained on rollouts of state transitions generated by a dynamics model. However, once learned, a model-based planning algorithm called Model Path Predictive Integral (MPPI) (**<empty citation>**) is used to find the optimal skill sequence in the skill distribution p_ψ, resulting in executable actions through the policy π_θ.

citation

[1] Rather, the sum of the state and a correction term, resulting from a separate network (section 4.2), but that is not relevant for the basic idea of the policy.

3.2. Learning the dynamic model

One way of having continually adapting control for soft robots, is by implementing a (traditional) control scheme that uses a dynamic model (either forward or inverse) of the robot and then to incrementally update that model. The idea of learning a dynamic model for soft robots has advantages even outside the context of lifelong adaptation: many soft robots have such a large amount of degrees of freedom and such complex and nonlinear dynamics, that analytically deriving a dynamic model is both cumbersome and often insufficiently accurate (Gillespie et al., 2018; Giorelli et al., 2015; Kim et al., 2021; Thuruthel et al., 2019).

Blackbox methods also allow for learning both forward and inverse dynamic models and can thus be used in several control schemes. When learning a forward model, the robot's actions and resulting states can be used as the inputs and outputs, respectively, this is so-called *direct modeling* (Nguyen-Tuong & Peters, 2011b). In the case of learning inverse dynamic models, such direct approaches might not always be applicable, due to the multiple solutions that exist for the inverse dynamics of robots with redundant degrees of freedom. Instead, an *indirect-modeling* approach can be taken, where the model is trained to minimize the error of the feedback controller (Nguyen-Tuong & Peters, 2011b).

The form that the dynamic model takes can vary. For example, in Gillespie et al. (2018), a neural network is trained to predict the next state of the robot, given the current state and a control action, the analytical gradients of these outputs w.r.t. to the inputs are made available during the process of backpropagation and can be used in matrices A and B to form a more interpretable state-space system. In Thuruthel et al. (2019) on the other hand, a forward model is learned for open-loop control, using the control action, the current state, and the previous state as input, and the next state as output to train a recurrent neural network. This forward model is kept in its more abstract blackbox form as a recurrent neural network (RNN), and is used in combination with trajectory optimization to generate trajectories in open-loop control.

Since the lifelong adapting nature of the techniques is embedded in the dynamic model, the choice of control scheme in which the learned dynamic model is used can vary from implementation to implementation. To illustrate, the aforementioned matrices A and B from Gillespie et al. (2018) are used in a model predictive control setting, while the generated trajectories from Thuruthel et al. (2019) are used as samples to train a blackbox control policy (the dynamic model is in essence an intermediary step in order to create the blackbox policy mentioned in section 3.1). More examples several blackbox dynamic models and their control scheme can be found in the literature.

Nazeer et al. (2023) represent a forward dynamic model of the soft robotic arm as an RNN model that uses the two preceding states and three preceding control actions to predict the next state. The model is then used with the previously discussed MLP policy (section 3.1).

Lu et al. (2020) was discussed in section 3.1 where it was briefly mentioned that training rollouts for the policies were generated using a dynamic model. This model is in fact also a data-driven model, being an ensemble of neural networks and predicting the next state s' based on the current state s and a control action yielded by the policy π_θ.

In and Nguyen-Tuong and Peters (2011a), the inverse dynamic model is represented by a Gaussian process (a type of probabilistic model) and is learned through Local Gaussian Process Regression (Nguyen-Tuong et al., 2008, 2009). The models are employed in a feedforward control scheme, supplemented by a feedback term that adjusts the control action outputted by the dynamic model.

redundant references?

Explain Gijsberts and Metta (2011)

Romeres et al. (2016) represent an inverse dynamic model of a robotic arm as a Gaussian process, but also experiment with using the arm's rigid body dynamics in several ways to enhance the model's performance. By incorporating the information from the robot's physical parameters into both the mean function and the covariance kernel of the Gaussian process,

they develop (a) semi-parametric model(s) that are used in a feedforward manner where a feedback controller adds small corrections to the control actions outputted by the dynamic model.

Contrary to blackbox policy learning methods, the adaptation to new tasks is not included in the learning process of the dynamic model, but is entirely dependent on the control policy. In case control policies have been defined in advance for several tasks, these task settings might serve as a test to verify that the dynamic model generalizes well over these different settings. In situations where tasks are not known in advance, a control scheme such as an incrementally updated (inverse) dynamics model in some form of a feedback control loop might serve for simple applications (Nguyen-Tuong & Peters, 2011b). For more complex tasks, a more intricate control system might be warranted.

rethink order of paragraphs

3.3. Incremental learning

In section 3.1 and 3.2 an overview was provided of how blackbox machine learning techniques can be used to model either control policies or dynamic models (to be used in control policies). The choice of machine learning method can be important when it comes to the desired incremental nature of the learning techniques. To illustrate, consider these two main strategies in continually adaptive control:

1. All training is performed offline (after gathering sufficient data) and the robot is now completely capable of immediately adapting to changing dynamics it will encounter during its operational lifespan.

2. Training is performed incrementally as data comes in, which can potentially be done in real-time.

In both scenarios, the control of the robot would be continually adaptive. However, only strategy 2 aligns with the incremental objectives of the adaptive control challenge. In addition, the complex dynamics and many degrees of freedom of soft robots might make it difficult to accurately represent all possible degradations in dynamics (such as damages) or state-space configurations of the robot, potentially causing an imbalance in the training dataset of strategy 1. Incrementally learning to adapt the control policy or dynamic model is thus far more desirable, since it can cope with unforeseen circumstances.

Not all the previously discussed methods are - or have the possibility to be - implemented in an incremental manner. As an example, the two policies from Zhou et al. (2022) mentioned in section 3.1 are trained sequentially: one online and the other offline. Each of these policies required different training/optimization algorithms for their specific setting, in this case Maximum a Posteriori Policy Optimization (MPO) (Abdolmaleki et al., 2018) for the online interaction setting, and Critic Regularized Regression (CRR) (Wang et al., 2020) for the offline distillation setting (see **??**).

reference chapter

To learn the dynamic model from Gijsberts and Metta (2011), the authors implement Ridge Regression (a linear regression method that aims at keeping the parameters as small as possible) incrementally, whereas normally it is used in a batch-learning setting. They achieve this by employing a modified numerical approach that is based on the QR algorithm (**<empty citation>** a technique in linear algebra that can process new information incrementally using Givens rotations (a process that is particularly efficient for online learning scenarios) (**<empty citation>**). This enables real-time learning as new data comes in without recalculating the full model.

citation [17] from Gijsberts and Metta (2011)

citation [16] in Gijsberts and Metta (2011)

Lu et al. (2020) implement both offline and online training, with the potential for the entire method to be implemented online. In fact, it is well known that SAC - the algorithm used for updating the skill-space distribution and the policy - is very suitable for online updates. When it

comes to the dynamic model, they mention that is incrementally updated through interactions with the environment, but do not provide any details on the algorithm used for training the dynamic model.

In Nguyen-Tuong and Peters (2011a) the authors not only update their model incrementally, but they also manage to do it in real-time. This is achieved by using incremental Gaussian process regression algorithm, which uses only a select number of data points to fit a model. In addition, they introduce an efficient way to incrementally update the dictionary of datapoints in a way that it maintains the most informative points for a given budget. These two techniques are key to reducing the computational demand and allowing for real-time updates.

3.4. Summary and conclusions

In Table 3.1 an overview of the discussed methods can be found.

Reference	Dynamic model	Control policy	Increme
Gillespie et al. (2018)	SS-matrices	–	✓
Gijsberts and Metta (2011)	Linear model in random feature space	–	✓
Thuruthel et al. (2019)[2]	RNN	Open-loop + trajectory optimization	–
Lu et al. (2020)	NN ensemble	MPPI + NN	✓
Nguyen-Tuong and Peters (2011a)	Gaussian process	Feedforward	✓
Romeres et al. (2016)	Gaussian process	feedforward + feedback	✓
Nazeer et al. (2023)	RNN	MLP	[3]
Thuruthel et al. (2019)[4]	–	FNN	–
Alessi et al. (2023)	–	FNN	–
Zhou et al. (2022)	–	Probabilistic model	✓

Table 3.1: An overview blackbox methods and their ability to be implemented incrementally. For dynamic model learning methods, the control policy specifies the control scheme in which the model is used.

add LU2019 to chapter, footnotes, checkmarks, table size, add the training algorithm (eg. PPO)?

[1] Used for generating training samples
[2] The incremental nature of the method is achieved by a third network, not by the dynamics model or control policy
[3] After training on generated samples

4

Learning architectures

4.1. Dual architectures

(Lu et al., 2019, 2020; Mallya et al., 2018; Schwarz et al., 2018; Shin et al., 2017; Sprechmann et al., 2018).

4.2. Architectural expansions

(Chatzilygeroudis et al., 2016; Gillespie et al., 2018; Nazeer et al., 2023).

5

Storage and retrieval

(Aljundi et al., 2016; Mouret & Clune, 2015; Nguyen-Tuong & Peters, 2011a; Schulman et al., 2017; Shin et al., 2017; Sprechmann et al., 2018; Wang et al., 2022; Xie & Finn, 2021).

6

Discussion and conclusion

Todo list

Bibliography

Abdolmaleki, A., Springenberg, J. T., Tassa, Y., Munos, R., Heess, N., & Riedmiller, M. (2018). Maximum a posteriori policy optimisation. http://arxiv.org/abs/1806.06920

Alessi, C., Hauser, H., Lucantonio, A., & Falotico, E. (2023). Learning a controller for soft robotic arms and testing its generalization to new observations, dynamics, and tasks. *2023 IEEE International Conference on Soft Robotics (RoboSoft)*, 1–7. https://doi.org/10.1109/RoboSoft55895.2023.10121988

Aljundi, R., Chakravarty, P., & Tuytelaars, T. (2016). Expert gate: Lifelong learning with a network of experts. http://arxiv.org/abs/1611.06194

Annaswamy, A. M., & Fradkov, A. L. (2021, January). A historical perspective of adaptive control and learning. https://doi.org/10.1016/j.arcontrol.2021.10.014

Chatzilygeroudis, K., Cully, A., & Mouret, J.-B. (2016). Towards semi-episodic learning for robot damage recovery. http://arxiv.org/abs/1610.01407

Gijsberts, A., & Metta, G. (2011). Incremental learning of robot dynamics using random features. *2011 IEEE International Conference on Robotics and Automation*, 951–956. https://doi.org/10.1109/ICRA.2011.5980191

Gillespie, M. T., Best, C. M., Townsend, E. C., Wingate, D., & Killpack, M. D. (2018). Learning nonlinear dynamic models of soft robots for model predictive control with neural networks. *2018 IEEE International Conference on Soft Robotics (RoboSoft)*, 39–45. https://doi.org/10.1109/ROBOSOFT.2018.8404894

Giorelli, M., Renda, F., Calisti, M., Arienti, A., Ferri, G., & Laschi, C. (2015). Learning the inverse kinetics of an octopus-like manipulator in three-dimensional space. *Bioinspiration & Biomimetics*, *10*, 035006. https://doi.org/10.1088/1748-3190/10/3/035006

Kim, D., Kim, S.-H., Kim, T., Kang, B. B., Lee, M., Park, W., Ku, S., Kim, D., Kwon, J., Lee, H., Bae, J., Park, Y.-L., Cho, K.-J., & Jo, S. (2021). Review of machine learning methods in soft robotics (G. Gu, Ed.). *PLOS ONE*, *16*, e0246102. https://doi.org/10.1371/journal.pone.0246102

Landau, I. D., Lozano, R., M'Saad, M., & Karimi, A. (2011). *Adaptive control*. Springer London. https://doi.org/10.1007/978-0-85729-664-1

Lesort, T., Lomonaco, V., Stoian, A., Maltoni, D., Filliat, D., & Díaz-Rodríguez, N. (2020). Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, *58*, 52–68. https://doi.org/10.1016/j.inffus.2019.12.004

Lu, K., Grover, A., Abbeel, P., & Mordatch, I. (2020). Reset-free lifelong learning with skill-space planning. http://arxiv.org/abs/2012.03548

Lu, K., Mordatch, I., & Abbeel, P. (2019). Adaptive online planning for continual lifelong learning. http://arxiv.org/abs/1912.01188

Mallya, A., Davis, D., & Lazebnik, S. (2018). Piggyback: Adapting a single network to multiple tasks by learning to mask weights. http://arxiv.org/abs/1801.06519

Mouret, J.-B., & Clune, J. (2015). Illuminating search spaces by mapping elites. http://arxiv.org/abs/1504.04909

Nazeer, M. S., Bianchi, D., Campinoti, G., Laschi, C., & Falotico, E. (2023). Policy adaptation using an online regressing network in a soft robotic arm. *2023 IEEE International Con-*

ference on Soft Robotics (RoboSoft), 1–7. https://doi.org/10.1109/RoboSoft55895.2023.10121927

Nguyen-Tuong, D., & Peters, J. (2011a). Incremental online sparsification for model learning in real-time robot control. *Neurocomputing*, *74*, 1859–1867. https://doi.org/10.1016/j.neucom.2010.06.033

Nguyen-Tuong, D., & Peters, J. (2011b). Model learning for robot control: A survey. *Cognitive Processing*, *12*, 319–340. https://doi.org/10.1007/s10339-011-0404-1

Nguyen-Tuong, D., Peters, J., & Seeger, M. (2008). Local gaussian process regression for real time online model learning and control. *NIPS'08: Proceedings of the 21st International Conference on Neural Information Processing Systems*, 1193–1200. https://doi.org/abs/10.5555/2981780.2981929

Nguyen-Tuong, D., Seeger, M., & Peters, J. (2009). Model learning with local gaussian process regression. *Advanced Robotics*, *23*, 2015–2034. https://doi.org/10.1163/016918609X12529286896

Romeres, D., Zorzi, M., Camoriano, R., & Chiuso, A. (2016). Online semi-parametric learning for inverse dynamics modeling. http://arxiv.org/abs/1603.05412

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. http://arxiv.org/abs/1707.06347

Schwarz, J., Luketina, J., Czarnecki, W. M., Grabska-Barwinska, A., Teh, Y. W., Pascanu, R., & Hadsell, R. (2018). Progress & compress: A scalable framework for continual learning. http://arxiv.org/abs/1805.06370

Shin, H., Lee, J. K., Kim, J., & Kim, J. (2017). Continual learning with deep generative replay. http://arxiv.org/abs/1705.08690

Sprechmann, P., Jayakumar, S. M., Rae, J. W., Pritzel, A., Badia, A. P., Uria, B., Vinyals, O., Hassabis, D., Pascanu, R., & Blundell, C. (2018). Memory-based parameter adaptation. http://arxiv.org/abs/1802.10542

Thuruthel, T. G., Falotico, E., Renda, F., & Laschi, C. (2019). Model-based reinforcement learning for closed-loop dynamic control of soft robotic manipulators. *IEEE Transactions on Robotics*, *35*, 124–134. https://doi.org/10.1109/TRO.2018.2878318

Wang, Z., Chen, C., & Dong, D. (2022). Lifelong incremental reinforcement learning with online bayesian inference. *IEEE Transactions on Neural Networks and Learning Systems*, *33*, 4003–4016. https://doi.org/10.1109/TNNLS.2021.3055499

Wang, Z., Novikov, A., Zolna, K., Springenberg, J. T., Reed, S., Shahriari, B., Siegel, N., Merel, J., Gulcehre, C., Heess, N., & de Freitas, N. (2020). Critic regularized regression. http://arxiv.org/abs/2006.15134

Xie, A., & Finn, C. (2021). Lifelong robotic reinforcement learning by retaining experiences. http://arxiv.org/abs/2109.09180

Zhou, W., Bohez, S., Humplik, J., Abdolmaleki, A., Rao, D., Wulfmeier, M., Haarnoja, T., & Heess, N. (2022). Forgetting and imbalance in robot lifelong learning with off-policy data. http://arxiv.org/abs/2204.05893